

Guía Pedagógica: Día 11 - Deferrable Views (Carga Diferida)

Hoy aprenderemos sobre una de las características más potentes de Angular para mejorar el rendimiento: los **Deferrable Views** (@defer). Esto nos permite retrasar la carga de ciertos componentes pesados (como reproductores de video o secciones de comentarios) hasta que sean estrictamente necesarios, mejorando nuestras métricas de *Web Vitals*.

Contexto Pedagógico: ¿Por qué diferir la carga?

En la página de detalles de una película, podríamos tener información crítica (título, sinopsis) y otra secundaria (el tráiler, lista de actores secundarios, o comentarios).

- **El problema:** Si cargamos todo de inmediato, el tiempo de carga inicial (*First Contentful Paint*) y el peso del paquete principal de JavaScript se incrementan, penalizando la experiencia del usuario y el SEO.
- **La solución (@defer):** Angular nos permite indicar de manera declarativa en el HTML qué partes deben cargarse "más tarde" (por ejemplo, cuando el usuario hace scroll hacia ellas). Angular se encarga del *Lazy Loading* automático de esos componentes.

Paso 1: Identificando los Bloques Pesados

Antes de crear el componente de tráiler, necesitamos un método en nuestro servicio para obtener los videos de una película.

Abre **src/app/core/services/movie.service.ts** y añade este método al final de la clase:

```
/**
 * Obtiene los videos (tráilers, teasers, etc.) de una película.
 * @param id ID de la película en TMDB
 */
getMovieVideos(id: string | number) {
  return this.http.get<{ results: Array<{ key: string; site:
string; type: string; name: string }> }>(
    `${this.apiUrl}/movie/${id}/videos`
  );
}
```

Paso 2: Crear el Pipe de Seguridad (SafePipe)

Angular bloquea por seguridad las URLs dinámicas en iframes para prevenir ataques XSS. Necesitamos un *Pipe* que le diga a Angular: "confío en esta URL, déjala pasar".

Crea el archivo **src/app/shared/pipes/safe.pipe.ts**:

```
import { Pipe, PipeTransform } from '@angular/core';
import { DomSanitizer, SafeResourceUrl } from '@angular/platform-browser';

/**
 * PIPE DE SEGURIDAD (Safe Pipe)
 * Solo se debe usar con URLs controladas (como las de YouTube).
 */
@Pipe({
  name: 'safe',
  standalone: true
})
export class SafePipe implements PipeTransform {
  constructor(private sanitizer: DomSanitizer) {}

  transform(url: string): SafeResourceUrl {
    return this.sanitizer.bypassSecurityTrustResourceUrl(url);
  }
}
```

Paso 3: Crear el Componente MovieTrailer

Este es el componente "pesado" que queremos diferir. Carga un iframe de YouTube con el tráiler oficial.

Crea el **archivo src/app/features/movie-details/components/movie-trailer/movie-trailer.ts**:

```

import { Component, inject, Input, OnInit, signal } from '@angular/core';
import { CommonModule } from '@angular/common';
import { MovieService } from '../../../core/services/movie.service';
import { SafePipe } from '../../../shared/pipes/safe.pipe';

@Component({
  selector: 'app-movie-trailer',
  standalone: true,
  imports: [CommonModule, SafePipe],
  template: `
    @if (trailerKey()) {
      <div class="trailer-wrapper">
        <iframe
          [src]="https://www.youtube.com/embed/' + trailerKey() | safe"
          title="Tráiler Oficial"
          frameborder="0"
          allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope; picture-in-picture"
          allowfullscreen
          class="trailer-iframe">
        </iframe>
      </div>
    } @else {
      <div class="no-trailer">
        <span class="no-trailer-icon">🎬</span>
        <p>No hay tráiler disponible para esta película.</p>
      </div>
    }
  `,
  styleUrls: ['./movie-trailer.css']
})
export class MovieTrailer implements OnInit {
  private movieService = inject(MovieService);

  @Input() movieId!: number;
  trailerKey = signal<string | null>(null);

  ngOnInit(): void {
    if (this.movieId) {
      this.movieService.getMovieVideos(this.movieId).subscribe({
        next: (data) => {
          // Buscamos el tráiler oficial de YouTube
          const trailer = data.results.find(
            v => v.site === 'YouTube' && (v.type === 'Trailer' || v.type === 'Teaser')
          );
          if (trailer) {
            this.trailerKey.set(trailer.key);
          }
        }
      });
    }
  }
}

```

3.2 Crear los estilos del Tráiler

Crea el archivo de estilos **src/app/features/movie-details/components/movie-trailer/movie-trailer.css**:

```

.trailer-wrapper {
  position: relative;
  width: 100%;
  padding-bottom: 56.25%; /* Aspect Ratio 16:9 */
  border-radius: 12px;
  overflow: hidden;
  background: #000;
}
.trailer-iframe {
  position: absolute;
  top: 0; left: 0;
  width: 100%; height: 100%;
  border: none;
}
.no-trailer {
  height: 300px;
  background: #rgba(255, 255, 255, 0.05);
  border-radius: 12px;
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: center;
  gap: 1rem;
  color: #888;
}
.no-trailer-icon { font-size: 3rem; }

```

Paso 4: Aplicar @defer en la Vista de Detalles

Ahora viene lo más importante. Abrimos `src/app/features/movie-details/movie-details.html` y envolvemos las secciones secundarias con `@defer`.

4.1 Tráiler con @defer (on viewport)

Añadimos una nueva sección de tráiler justo después de la sinopsis. El tráiler (y todo su JavaScript + iframe de YouTube) solo se descargará cuando el usuario haga scroll hasta esa zona.

4.2 Elenco con @defer (on viewport)

El elenco de actores ya existe en nuestro HTML. Lo envolvemos con @defer para que también se cargue de forma diferida.

El HTML completo queda así (solo se muestran las secciones modificadas dentro del bloque @if):

```
<section class="movie-info">
  <h2>Sinopsis</h2>
  <p class="overview">{{ data.details.overview }}</p>
</section>

<!-- CARGA DIFERIDA: EL Tráiler (on viewport) -->
<section class="trailer-section">
  <h2>Tráiler Oficial</h2>

  @defer (on viewport) {
    <!-- Este componente solo se descarga cuando esta sección entra en pantalla -->
    <app-movie-trailer [movieId]="data.details.id"></app-movie-trailer>
  } @placeholder (minimum 500ms) {
    <!-- Lo que se muestra ANTES de que se cumpla la condición -->
    <div class="trailer-placeholder">
      <p>🎬 Haz scroll para ver el tráiler...</p>
    </div>
  } @loading {
    <!-- Lo que se muestra DURANTE la descarga del componente -->
    <div class="defer-spinner">Cargando reproductor...</div>
  } @error {
    <!-- Si la red falla al intentar descargar el componente -->
    <p class="defer-error">Error al cargar el reproductor de video.</p>
  }
</section>

<!-- CARGA DIFERIDA: EL Elenco (on viewport) -->
<section class="movie-cast">
  <h2>Elenco Principal</h2>

  @defer (on viewport) {
    <div class="cast-track">
      @for (actor of data.credits.cast | slice:0:15; track actor.id) {
        <app-cast-card [actor]="actor"></app-cast-card>
      } @empty {
        <p>No hay información del elenco.</p>
      }
    </div>
  } @placeholder {
    <div class="cast-placeholder">
      <p>👤 Haz scroll para ver el elenco...</p>
    </div>
  } @loading {
    <div class="defer-spinner">Cargando elenco...</div>
  }
</section>
```

Paso 5: Entendiendo los Triggers (Desencadenadores)

El bloque @defer admite varias condiciones (triggers) para iniciar la descarga:

1. on viewport: Cuando el elemento entra en la pantalla (usa un *Intersection Observer* nativo por detrás). Es el que usamos aquí.
2. on interaction: Cuando el usuario hace clic o toca el área del placeholder.
3. on hover: Cuando el cursor pasa por encima del área.
4. on timer(5s): Carga automática después de un tiempo definido (ej. 5 segundos).
5. when condicionBoolean: Carga imperativa basada en una señal o variable booleana.

Se pueden combinar: @defer (on interaction; on hover) carga cuando el usuario hace clic o pasa el ratón por encima.

Paso 6: Estilos para los Placeholders (CSS)

Añade estos estilos al final de **src/app/features/movie-details/movie-details.css**:

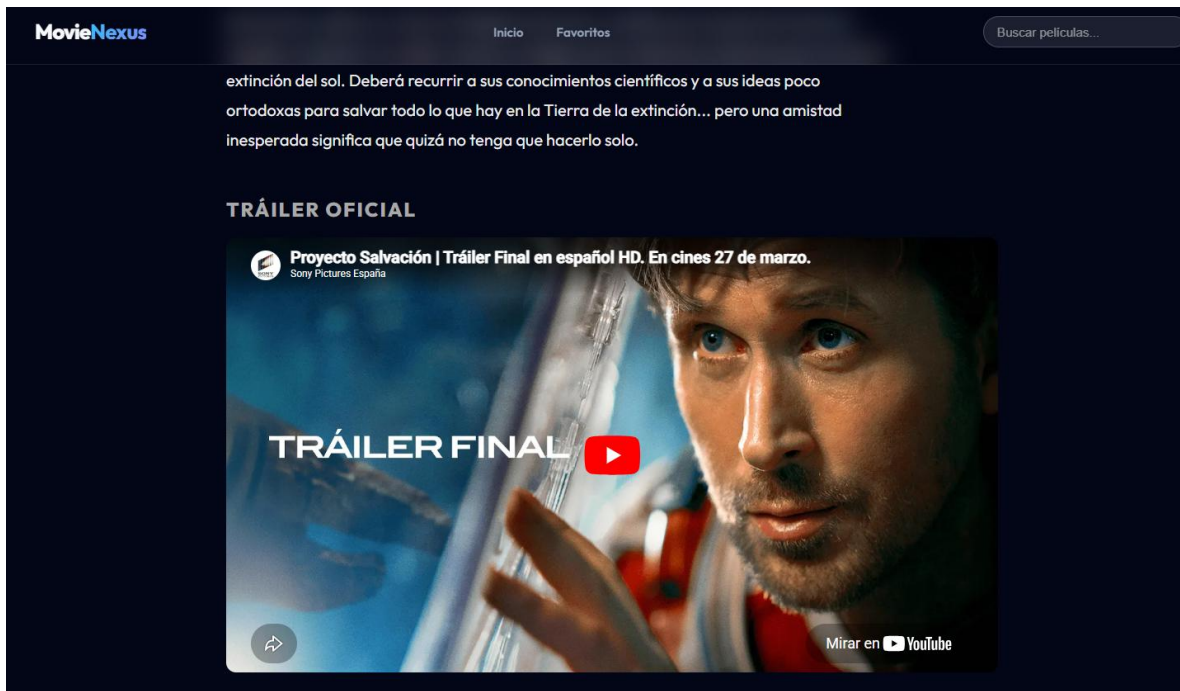
```
/* === Sección de Tráiler === */
.trailer-section {
  margin-bottom: 3rem;
}

.trailer-section h2 {
  font-size: 1.5rem;
  color: #aaa;
  text-transform: uppercase;
  letter-spacing: 2px;
  margin-bottom: 1rem;
}

/* Placeholders para Deferrable Views */
.trailer-placeholder {
  height: 400px;
  background: rgba(255, 255, 255, 0.03);
  display: flex;
  justify-content: center;
  align-items: center;
  border-radius: 12px;
  border: 1px dashed rgba(255, 255, 255, 0.15);
  color: #888;
  font-size: 1.1rem;
}

.cast-placeholder {
  height: 200px;
  background: rgba(255, 255, 255, 0.03);
  display: flex;
  justify-content: center;
  align-items: center;
  border-radius: 12px;
  border: 1px dashed rgba(255, 255, 255, 0.15);
  color: #888;
  font-size: 1.1rem;
}

.defer-spinner {
  display: flex;
  justify-content: center;
  padding: 2rem;
  color: #e50914;
  font-weight: bold;
}
```



Prueba de Aprendizaje (Checklist)

1. **¿Cuál es la diferencia principal entre `@defer` y usar un `@if` estándar para ocultar contenido?** (Respuesta: `@if` decide si el componente se renderiza o no en el DOM, pero el código fuente JavaScript del componente ya fue descargado en el bundle principal. `@defer` hace "Lazy Loading" a nivel de componente: el navegador ni siquiera descarga el JavaScript de esa parte hasta que se cumpla la condición, haciendo la carga inicial más rápida).
2. **¿Para qué sirve el bloque `@placeholder`?** (Respuesta: Muestra un contenido temporal que reserva el espacio en el DOM mientras se espera que se cumpla el trigger (ej. hacer scroll). Es vital para evitar "saltos" en la interfaz o Layout Shifts).
3. **¿Qué ocurre internamente si especificas `(on viewport)`?** (Respuesta: Angular utiliza la API de Intersection Observer del navegador para detectar cuándo el placeholder se vuelve visible en la pantalla y, en ese preciso momento, inicia la descarga del componente).

Control de Versiones

Finaliza el día guardando tu progreso:

```
git add .
git commit -m "feat: implementar carga diferida (@defer) en la vista de detalles para optimizar rendimiento"
git push origin main
```